

Đặc tả Yêu cầu Phần mềm (SRS)

Fintech Core Service Layer — Hệ thống lõi ngân hàng kiểu T24

Phiên bản tài liệu	1.0
Ngày	2026-09-15
Trạng thái hệ thống	Demo / Proof-of-concept — chưa dùng cho production
Kho mã nguồn	d:\Demo MBBank

1. Giới thiệu

1.1 Mục đích

Tài liệu này đặc tả yêu cầu chức năng và phi chức năng của **Fintech Core Service Layer** — một hệ thống mô phỏng lõi ngân hàng (core banking system) theo kiến trúc và nghiệp vụ tương tự Temenos T24, được xây dựng làm dự án demo/học tập. Tài liệu ghi lại **đúng những gì đã cài đặt và đã kiểm thử bằng dữ liệu thật**, không phải đặc tả dự định trước khi code.

1.2 Phạm vi

Hệ thống cung cấp:

- REST API quản lý nghiệp vụ ngân hàng bán lẻ: khách hàng, tài khoản, thẻ, giao dịch, tiền gửi có kỳ hạn, cho vay, cảnh báo gian lận, tỷ giá/quy đổi ngoại tệ.
- Sổ cái kế toán kép (General Ledger) ghi nhận tự động mọi giao dịch tiền.
- Cơ chế kiểm soát kép (Maker-Checker) với xác thực đăng nhập thật (JWT).
- Batch xử lý cuối ngày (Close of Business) chạy tự động theo lịch (cron) hoặc thủ công.
- Giao diện web quản trị nội bộ ("FinCore Console" tại <http://localhost:3000/>) và tài liệu API tự sinh (Swagger UI tại <http://localhost:3000/docs>).

Ngoài phạm vi (chưa cài đặt): đa chi nhánh/đa công ty, đa tiền tệ đầy đủ (chỉ hỗ trợ quy đổi 2 chiều VND ↔ 1 ngoại tệ), refresh token/thu hồi phiên đăng nhập, giao diện khách hàng (mobile/internet banking), báo cáo quản trị/regulatory reporting.

1.3 Từ viết tắt và thuật ngữ

THUẬT NGỮ	Ý NGHĨA
GL	General Ledger — Sổ cái kế toán kép
COB	Close of Business — batch xử lý cuối ngày

THUẬT NGỮ	Ý NGHĨA
Maker-Checker	Nguyên tắc kiểm soát kép: người nhập lệnh (maker) khác người duyệt lệnh (checker)
NPL	Non-Performing Loan — nợ xấu (quá hạn > 90 ngày, chuẩn phân loại phổ biến kiểu Basel)
RBAC	Role-Based Access Control — phân quyền theo vai trò
JWT	JSON Web Token — token xác thực đăng nhập
Idempotency-Key	Header HTTP đảm bảo 1 yêu cầu ghi tiền không bị lặp khi client gọi lại
As-of	Truy vấn giá trị hiệu lực tính đến 1 thời điểm (vd tỷ giá as-of ngày X)
Sổ tiết kiệm / Term Deposit	Tiền gửi có kỳ hạn

1.4 Tổng quan tài liệu

Mục 2 mô tả tổng quan hệ thống và kiến trúc. Mục 3 mô tả mô hình dữ liệu. Mục 4 đặc tả chi tiết yêu cầu chức năng theo từng module nghiệp vụ, gồm cả quy tắc nghiệp vụ đã kiểm thử thật. Mục 5 nêu yêu cầu phi chức năng. Mục 6 liệt kê giao diện ngoài (API, Web Console). Mục 7 là phụ lục.

2. Mô tả tổng quan

2.1 Bối cảnh sản phẩm

Hệ thống là 1 service layer độc lập (không phụ thuộc core banking thật nào khác), kết nối trực tiếp PostgreSQL (Supabase) qua driver **pg** thuần — không dùng ORM, không dùng framework ngoài Express. Toàn bộ business logic (tính lãi, hạch toán GL, kiểm soát kép, phân loại nợ xấu...) nằm trong tầng Service, không đẩy xuống database (trừ ràng buộc toàn vẹn cơ bản: FK, CHECK, UNIQUE).

2.2 Kiến trúc phân lớp

Request → Router → Middleware (validate/auth/idempotency) → Controller → Service → Repository → PostgreSQL

LỚP	TRÁCH NHIỆM
Routes (<code>src/routes/</code>)	Khai báo endpoint, gắn middleware validate/requireAuth/requireIdempotencyKey
Middleware (<code>src/middleware/</code>)	<code>validate</code> (zod), <code>requireAuth</code> (JWT), <code>requireIdempotencyKey</code> , <code>errorHandler</code>
Controllers (<code>src/controllers/</code>)	Chuyển đổi HTTP request/response, không chứa business logic
Services (<code>src/services/</code>)	Toàn bộ business rule, transaction boundary (<code>withTransaction</code>)

LỚP	TRÁCH NHIỆM
Repositories (<code>src/repositories/</code>)	Câu lệnh SQL thuần, không chứa business logic
Validators (<code>src/validators/</code>)	Schema zod cho từng endpoint

2.3 Công nghệ sử dụng

THÀNH PHẦN	CÔNG NGHỆ
Ngôn ngữ / Runtime	TypeScript (strict), Node.js, chạy qua <code>tsx watch</code> (dev)
Web framework	Express 4
Database	PostgreSQL (Supabase), driver <code>pg</code> (không ORM)
Validation	Zod (schema-first, coercion cho query/body)
Xác thực	<code>jsonwebtoken</code> (JWT) + <code>bcryptjs</code> (hash mật khẩu)
Batch scheduler	<code>node-cron</code>
Tài liệu API	<code>swagger-ui-express</code> tự sinh từ <code>src/config/openapi.ts</code>
Giao diện quản trị	1 trang HTML/CSS/JS thuần (<code>public/index.html</code>), không framework FE

2.4 Đối tượng người dùng

VAI TRÒ	MÔ TẢ	THAO TÁC CHÍNH
Maker (teller)	Nhân viên nhập lệnh nghiệp vụ	Nộp yêu cầu vào hàng đợi chờ duyệt
Checker (supervisor)	Nhân viên phê duyệt	Duyệt/từ chối yêu cầu của maker khác
Both/Admin	Vừa nhập vừa duyệt (không tự duyệt lệnh của chính mình)	Cả hai thao tác trên
Khách hàng (customers)	Đối tượng nghiệp vụ (không đăng nhập hệ thống này)	Không thao tác trực tiếp — mọi thao tác thay mặt khách hàng do maker/checker thực hiện

2.5 Ràng buộc thiết kế

- Schema gốc 11 bảng nghiệp vụ (`db/schema.postgres.sql`) **giữ nguyên cấu trúc** trong suốt quá trình mở rộng; mọi tính năng mới thêm bằng **migration bổ sung** (bảng mới hoặc `ALTER TABLE ADD COLUMN nullable`), không sửa đổi phá vỡ cấu trúc gốc.
- Toàn bộ migration **an toàn chạy lại nhiều lần** (`IF NOT EXISTS` , `DO $$ EXCEPTION WHEN duplicate_object$$ , ON CONFLICT DO NOTHING`).
- Không dùng ORM — mọi câu lệnh SQL viết tay trong tầng Repository, dùng placeholder dạng `:name` tự chuyển sang `$1..$n` .
- Tiền tệ nền (base/functional currency) của toàn hệ thống là **VND** — sổ cái không có chiều tiền tệ, mọi bút toán GL đều tính theo VND-equivalent.

2.6 Giả định và phụ thuộc

- Giả định 1 tài khoản (`account_id`) chỉ có 1 loại tiền tệ cố định suốt vòng đời.
- Giả định mỗi khoản vay/sổ tiết kiệm gắn với đúng 1 tài khoản thanh toán để giải ngân/nhận trả góp/nhận lãi.
- Phụ thuộc PostgreSQL hỗ trợ `GENERATED ALWAYS AS IDENTITY` , `JSONB` , kiểu `ENUM` , transaction với `FOR UPDATE` row-lock.

3. Mô hình dữ liệu

3.1 Nhóm bảng nghiệp vụ gốc (11 bảng — `db/schema.postgres.sql`)

BẢNG	VAI TRÒ	CỘT MỞ RỘNG THÊM SAU NÀY
<code>customers</code>	Khách hàng (Party)	—
<code>account_types</code>	Danh mục loại tài khoản (lãi suất mặc định)	—
<code>accounts</code>	Tài khoản thanh toán/tiết kiệm không kỳ hạn	—
<code>cards</code>	Thẻ ngân hàng gắn với account	—
<code>merchants</code>	Điểm bán (cho giao dịch loại <code>payment</code>)	—
<code>transaction_categories</code>	Danh mục giao dịch (income/spending/transfer/fee)	Seed thêm <code>Interest</code> (income)
<code>transactions</code>	Bảng trung tâm — mọi dòng tiền đều là 1 row ở đây	—
<code>loans</code>	Khoản vay	<code>account_id</code> (tài khoản giải ngân/trả góp)
<code>loan_payments</code>	Lịch trả góp	<code>transaction_id</code> (liên kết giao dịch trả góp thật)
<code>fraud_alerts</code>	Cảnh báo gian lận	—
<code>exchange_rates</code>	Tỷ giá theo ngày	—

3.2 Nhóm bảng mở rộng (migrations 001–007)

MIGRATION	BẢNG MỚI	MỤC ĐÍCH
<code>001_idempotency_keys.sql</code>	<code>idempotency_keys</code>	Chống double-post khi client gọi lại cùng request
<code>002_term_deposits.sql</code>	<code>term_deposits</code> , <code>term_deposit_postings</code>	Tiền gửi có kỳ hạn + lịch sử ghi nhận lãi

MIGRATION	BẢNG MỚI	MỤC ĐÍCH
<code>003_general_ledger.sql</code>	<code>gl_accounts</code> , <code>gl_entries</code> , <code>gl_posting_rules</code>	Sổ cái kế toán kép
<code>004_maker_checker.sql</code>	<code>app_users</code> , <code>auth_queue</code>	Người dùng nội bộ + hàng đội kiểm soát kép
<code>005_loan_account_link.sql</code>	(ALTER) <code>loans.account_id</code> , <code>loan_payments.transaction_id</code>	Nối khoản vay vào dòng tiền thật
<code>006_close_of_business.sql</code>	<code>cob_runs</code>	Log các lần chạy batch cuối ngày
<code>007_auth_login.sql</code>	(ALTER) <code>app_users.password_hash</code>	Đăng nhập thật (bcrypt)

3.3 Chart of Accounts (Hệ thống tài khoản kế toán)

MÃ	TÊN	LOẠI	SỐ DƯ BÌNH THƯỜNG
1000	Tiền mặt và ngân quỹ	Tài sản	Nợ
1100	Dư nợ cho vay khách hàng	Tài sản	Nợ
2000	Tiền gửi thanh toán khách hàng	Nợ phải trả	Có
2100	Tiền gửi có kỳ hạn khách hàng	Nợ phải trả	Có
4000	Thu nhập lãi cho vay	Thu nhập	Có
4100	Thu nhập phí dịch vụ	Thu nhập	Có
5000	Chi phí lãi tiền gửi	Chi phí	Nợ

4. Yêu cầu chức năng theo module

Mỗi module nêu: mô tả, quy tắc nghiệp vụ chính, và danh sách endpoint (tiền tố chung `/api/v1`). Hệ thống có **14 module chức năng, 55 endpoint** — xem danh sách khớp 1-1 với route trong mã nguồn tại Swagger UI: <http://localhost:3000/docs>.

4.1 Danh mục dùng chung (Reference Data) — FR-REF

Dữ liệu tham chiếu dùng chung cho các module khác — ít thay đổi, chủ yếu đọc.

- `account_types` : danh mục loại tài khoản kèm lãi suất mặc định, dùng khi mở `accounts` .
- `merchants` : điểm bán, dùng khi tạo `transactions` loại `payment` .
- `transaction_categories` : danh mục giao dịch (`category_group` ∈ {`income`, `spending`, `transfer`, `fee`}), bắt buộc mọi `transactions` phải gắn `category_id` .

METHOD	ENDPOINT
GET	<code>/account-types</code>

METHOD	ENDPOINT
POST	/account-types
GET	/merchants
GET	/transaction-categories

4.2 Quản lý khách hàng — FR-CUST

- Tạo, tìm kiếm (theo email/KYC), xem chi tiết, cập nhật thông tin, cập nhật trạng thái KYC (`pending` | `verified` | `rejected`).
- Hỗ trợ cây giới thiệu (`referred_by` tự tham chiếu `customers`), tra cứu danh sách được giới thiệu.

METHOD	ENDPOINT
POST	/customers
GET	/customers
GET	/customers/:id
PATCH	/customers/:id
PATCH	/customers/:id/kyc
GET	/customers/:id/referrals
GET	/customers/:id/accounts
POST	/customers/:id/loans (yêu cầu Idempotency-Key)

4.3 Quản lý tài khoản — FR-ACC

- Mở tài khoản (sinh số tài khoản tự động dạng `VNxxxxxxxxxxx`), gán `account_type_id` và `currency` .
- Đổi trạng thái theo **máy trạng thái hữu hạn** — `active` , `dormant` , `frozen` chuyển qua lại tự do lẫn nhau, cả 3 đều có thể chuyển sang `closed` ; `closed` là trạng thái kết thúc (terminal), không có transition nào đi ra khỏi nó (không thể mở lại).
- Lịch sử số dư (`balance-history`) suy diễn từ toàn bộ `transactions` liên quan (không lưu snapshot).

METHOD	ENDPOINT
POST	/accounts
GET	/accounts/:id
PATCH	/accounts/:id/status
GET	/accounts/:id/balance-history
POST	/accounts/:id/cards
GET	/accounts/:id/transactions
GET	/accounts/:id/term-deposits

4.4 Quản lý thẻ — FR-CARD

- Phát hành thẻ (`debit|credit|prepaid`) cho 1 account, đổi trạng thái (`active|blocked|expired`).

METHOD	ENDPOINT
GET	<code>/cards/:id</code>
PATCH	<code>/cards/:id/status</code>

4.5 Giao dịch (Transaction) — FR-TXN

Bảng `transactions` là **bảng trung tâm** — mọi module khác (tiền gửi, vay, FX) đều tạo ra 1 hoặc nhiều dòng ở bảng này để dòng tiền được nhất quán 1 nguồn sự thật.

Quy tắc nghiệp vụ:

- `txn_type` gồm: `deposit, withdrawal, transfer, payment, fee, interest`.
- `transfer` bắt buộc có `related_account_id`; các loại khác cấm có trường này.
- Account phải ở trạng thái `active`; `withdrawal` / `transfer` kiểm tra đủ số dư trước khi trừ.
- Ràng buộc tiền tệ (đã vá lỗi):** `currency` khai báo trong request PHẢI khớp tiền tệ thật của account — nếu không sẽ bị từ chối (`VALIDATION_ERROR`) thay vì âm thầm cộng/trừ sai số. Muốn đổi tiền tệ phải qua module FX (mục 4.9).
- Mọi giao dịch tạo mới đều tự động: (1) điều chỉnh `balance` của account liên quan, (2) hạch toán GL (mục 4.10), (3) chạy kiểm tra gian lận (mục 4.8).
- Đảo giao dịch (`reverse`):** không xoá bản gốc — đánh dấu `status='reversed'`, tạo 1 giao dịch mới mang hiệu ứng balance NGƯỢC LẠI, đồng thời hạch toán GL đảo Nợ/Có so với bút toán gốc (không lặp lại bút toán gốc).
- Toàn bộ POST tạo giao dịch bắt buộc header `Idempotency-Key`: request lặp lại cùng key + cùng nội dung trả lại response đã lưu; cùng key khác nội dung → lỗi `IDEMPOTENCY_KEY_CONFLICT`.

METHOD	ENDPOINT
POST	<code>/transactions</code> (yêu cầu <code>Idempotency-Key</code>)
GET	<code>/transactions/:id</code>
POST	<code>/transactions/:id/reverse</code>

4.6 Tiền gửi có kỳ hạn (Term Deposit / Sổ tiết kiệm) — FR-TD

Đây là module có logic tính lãi phức tạp nhất hệ thống. Mở sổ trích nộp `principal_amount` từ 1 account (ghi 1 giao dịch `withdrawal`, hạch toán Nợ 2000/Có 2100 — chuyển loại tiền gửi, không phải rút tiền mặt khỏi ngân hàng).

Tham số cấu hình mỗi sổ:

THAM SỐ	GIÁ TRỊ	Ý NGHĨA
<code>interest_method</code>	<code>simple</code> <code>compound</code>	Lãi đơn hoặc lãi kép (ghép hàng tháng)

THAM SỐ	GIÁ TRỊ	Ý NGHĨA
<code>payout_method</code>	<code>maturity</code> <code>monthly</code>	Trả lãi cuối kỳ hoặc hàng tháng
<code>day_count_convention</code>	<code>actual_365</code> <code>actual_360</code>	Cơ sở tính số ngày/năm
<code>early_withdrawal_rate</code>	mặc định 0.20%/năm	Lãi suất không kỳ hạn áp dụng khi rút trước hạn
<code>auto_renewal</code>	boolean	Tự động tái tục khi đáo hạn

Công thức:

- Lãi đơn: $I = P \times r\% \times \text{số_ngày} / \text{cơ_sở_năm}$
- Lãi kép (ghép tháng, tròn kỳ): $I = P \times ((1 + r\%/12)^{\text{số_tháng}} - 1)$
- Rút trước hạn (**case khó #1**): bất kể còn 1 ngày là đáo hạn, số **KHÔNG** được hưởng lãi suất cam kết — chỉ tính lãi đơn ở mức `early_withdrawal_rate` trên đúng số ngày thực gửi. Nếu là sổ trả lãi hàng tháng, các kỳ lãi đã trả trước đó khách vẫn giữ nguyên (không truy thu).
- Đáo hạn (**case khó #2**): nếu `auto_renewal=true` → tái tục sổ mới với gốc = gốc cũ + lãi vừa tính (lãi nhập gốc, không đi qua tài khoản thanh toán — hạch toán GL Nợ 5000/Có 2100); nếu `false` → tắt toán, cộng gốc+lãi vào tài khoản (Nợ 2100 (gốc) + Nợ 5000 (lãi) / Có 2000).
- Lãi dự thu (`accrued-interest`): tính đến hôm nay, **chỉ để tham khảo, không hạch toán** — khác với lãi thực nhận khi rút trước hạn.

METHOD	ENDPOINT
POST	<code>/term-deposits</code> (yêu cầu <i>Idempotency-Key</i>)
GET	<code>/term-deposits/:id</code>
GET	<code>/term-deposits/:id/accrued-interest</code>
GET	<code>/term-deposits/:id/postings</code>
POST	<code>/term-deposits/:id/interest-postings/monthly</code> (yêu cầu <i>Idempotency-Key</i>)
POST	<code>/term-deposits/:id/withdraw-early</code> (yêu cầu <i>Idempotency-Key</i>)
POST	<code>/term-deposits/:id/mature</code> (yêu cầu <i>Idempotency-Key</i>)

4.7 Cho vay (Loan) — FR-LOAN

- Tạo khoản vay: tự sinh lịch trả góp N kỳ theo phương pháp **dư nợ giảm dần, trả đều mỗi kỳ (French amortization)**; **giải ngân thật** vào `account_id` (cộng balance + hạch toán Nợ 1100/Có 2000). Bắt buộc header **`Idempotency-Key`** (đã bổ sung — trước đó là điểm chưa nhất quán so với các endpoint di chuyển tiền khác, xem mục 5.5).
- Trả góp: trừ tiền từ account, tách đúng bút toán gốc/lãi theo tỉ lệ đã lên lịch cho kỳ đó (Nợ 2000 tổng tiền / Có 1100 phần gốc + Có 4000 phần lãi). Trả hết kỳ cuối → `status='closed'`.
- Phân loại nợ xấu: xem COB (mục 4.12).

METHOD	ENDPOINT
GET	<code>/loans/:id</code>

METHOD	ENDPOINT
GET	<code>/loans/:id/payments</code>
POST	<code>/loans/:id/payments/:paymentId/pay</code> (yêu cầu <i>Idempotency-Key</i>)

(Endpoint tạo khoản vay `POST /customers/:id/Loans` — mục 4.2 — cũng yêu cầu *Idempotency-Key* vì có giải ngân tiền thật.)

4.8 Cảnh báo gian lận — FR-FRAUD

- Sau mỗi giao dịch, nếu `amount > FRAUD_AMOUNT_THRESHOLD` (mặc định 50.000.000đ, cấu hình qua `.env`) → tự động tạo `fraud_alerts` (`alert_type='LARGE_AMOUNT'`). `risk_score = min(100, round(amount / threshold × 50))` — tăng tuyến tính theo mức vượt ngưỡng, tối đa 100.
- Có thể tạo cảnh báo thủ công, cập nhật trạng thái theo máy trạng thái: từ `open` có thể chuyển thẳng sang `reviewing`, `closed_fp` hoặc `closed_confirmed`; từ `reviewing` chỉ chuyển tiếp sang `closed_fp` / `closed_confirmed`; `closed_fp` / `closed_confirmed` là trạng thái kết thúc.

METHOD	ENDPOINT
GET	<code>/fraud-alerts</code>
POST	<code>/fraud-alerts</code>
PATCH	<code>/fraud-alerts/:id/status</code>

4.9 Tỷ giá và Quy đổi ngoại tệ (FX) — FR-FX

- Tra cứu/tạo tỷ giá theo ngày (`exchange_rates`); tra cứu kiểu **as-of** (bản ghi gần nhất có `rate_date ≤` ngày yêu cầu).
- Quy đổi ngoại tệ giữa 2 account khác `currency`** (`/fx-transfers`): chỉ hỗ trợ cặp có 1 chân là VND (base currency của hệ thống — số cái không có chiều tiền tệ nên không hỗ trợ quy đổi chéo ngoại tệ ↔ ngoại tệ trực tiếp). Tạo 2 giao dịch riêng (rút ở account nguồn, nộp ở account đích theo số tiền đã quy đổi), hạch toán GL 1 cặp Nợ/Có cùng TK 2000 bằng giá trị VND-equivalent (net = 0 vì tiền vẫn thuộc nhóm "khách hàng gửi", chỉ đổi mẫu tiền tệ).

METHOD	ENDPOINT
GET	<code>/exchange-rates</code>
POST	<code>/exchange-rates</code>
POST	<code>/fx-transfers</code> (yêu cầu <i>Idempotency-Key</i>)

4.10 Sổ cái kế toán kép (General Ledger) — FR-GL

Bất biến kế toán kép: mọi nghiệp vụ tiền phát sinh đúng 1 cặp bút toán Nợ/Có cân bằng (cùng số tiền); tổng Nợ toàn hệ thống luôn bằng tổng Có.

- Giao dịch tài khoản thông thường (`deposit/withdrawal/transfer/payment/fee/interest`): hạch toán tự động theo bảng `gl_posting_rules` sau (ánh xạ `txn_type` → cặp tài khoản Nợ/Có mặc định):

TXN_TYPE	NỢ	CÓ	DIỄN GIẢI
deposit	1000	2000	Nộp tiền: Tiền mặt tăng / Tiền gửi khách hàng tăng
withdrawal	2000	1000	Rút tiền: ngược lại deposit
transfer	2000	2000	Chuyển khoản nội bộ — cùng 1 mã TK cả 2 chân, net = 0 (không phát sinh tiền mặt)
payment	2000	1000	Thanh toán merchant — xử lý như rút tiền
fee	2000	4100	Phí dịch vụ: trừ tiền khách hàng, ghi nhận thu nhập phí
interest	5000	2000	Trả lãi: ghi nhận chi phí lãi, cộng tiền khách hàng

- Nghiệp vụ có quy tắc riêng (mở/tắt toán sổ tiết kiệm, giải ngân/trả góp vay, quy đổi FX): hạch toán **tường minh** (không dùng quy tắc mặc định theo `txn_type`) vì bản chất kế toán khác nhau (chuyển loại tiền gửi ≠ rút tiền mặt).
- Đào giao dịch: hạch toán GL **đào chiều** so với bút toán gốc (không lặp lại).
- Báo cáo **Bảng cân đối thử** (`trial-balance`): tổng hợp Nợ/Có/số dư rỗng theo từng tài khoản kế toán, cờ `balanced` xác nhận tổng Nợ = tổng Có.
- Audit trail: tra cứu bút toán theo tài khoản kế toán (sổ cái chi tiết) hoặc theo `transaction_id` (biết 1 giao dịch cụ thể tạo ra bút toán GL nào).

METHOD	ENDPOINT
GET	<code>/gl/accounts</code>
GET	<code>/gl/trial-balance</code>
GET	<code>/gl/accounts/:code/entries</code>
GET	<code>/gl/transactions/:id/entries</code>

4.11 Đăng nhập, Phân quyền & Kiểm soát kép (Maker-Checker) — FR-AUTH

Đăng nhập (FR-AUTH-01):

- `app_users` (nhân sự nội bộ, khác `customers`) có `username`, `password_hash` (bcrypt), `role ∈ {maker, checker, both, admin}`.
- `POST /auth/login` xác thực, cấp JWT (hết hạn theo `JWT_EXPIRES_IN`, mặc định 8h). `GET /auth/me` trả thông tin user theo token.

Kiểm soát kép (FR-AUTH-02): mọi thao tác nhạy cảm đi qua hàng đợi `auth_queue` thay vì thực thi ngay:

- Maker gọi `POST /auth-queue` với `{operation_type, payload}` — `maker_id` lấy từ JWT đã xác thực, **không nhận qua body** (chống giả mạo danh tính). Payload được validate theo đúng schema của nghiệp vụ tương ứng ngay lúc nộp.
- Yêu cầu ở trạng thái `pending` — **chưa có hiệu lực, tiền chưa di chuyển**.
- Checker gọi `POST /auth-queue/:id/authorize` → hệ thống thực thi payload bằng đúng service layer thật (transaction/tài khoản/sổ tiết kiệm), lưu `result`.

4. 2 lớp chặn bắt buộc:

- **Chặn tự duyệt:** `checker_id === maker_id` → từ chối (`AUTH_QUEUE_SELF_AUTHORIZATION`).
- **Chặn theo vai trò (RBAC):** chỉ role `checker/both/admin` được authorize/reject; chỉ role `maker/both/admin` được enqueue — role `maker` đơn thuần **không thể phê duyệt dù không phải người nộp lệnh**.

5. Nếu thực thi lỗi sau khi đã "claim" (chuyển `pending→authorized`), hệ thống tự hoàn tác về `pending` để checker thử lại — không để hàng đợi kẹt ở trạng thái sai sự thật.

6. `operation_type` hỗ trợ: `CREATE_TRANSACTION` , `UPDATE_ACCOUNT_STATUS` , `OPEN_TERM_DEPOSIT` , `WITHDRAW_TERM_DEPOSIT_EARLY` , `MATURE_TERM_DEPOSIT` .

METHOD	ENDPOINT	XÁC THỰC
POST	<code>/auth/login</code>	—
GET	<code>/auth/me</code>	Bearer JWT
GET	<code>/users</code>	—
POST	<code>/auth-queue</code>	Bearer JWT (role maker/both/admin)
GET	<code>/auth-queue</code>	—
GET	<code>/auth-queue/:id</code>	—
POST	<code>/auth-queue/:id/authorize</code>	Bearer JWT (role checker/both/admin)
POST	<code>/auth-queue/:id/reject</code>	Bearer JWT (role checker/both/admin)

4.12 Batch cuối ngày (Close of Business — COB) — FR-COB

Mô phỏng batch COB thật của ngân hàng: **chạy tự động qua cron** (`COB_CRON_SCHEDULE` , mặc định `0 0 * *` * — 00:00 mỗi ngày) hoặc trigger thủ công qua API/UI. Mỗi lần chạy:

1. **Tự động đáo hạn** mọi `term_deposits` đang `active` có `maturity_date ≤` ngày xử lý (gọi lại đúng `termDepositService.mature`).
2. **Phân loại nợ xấu:** khoản vay `active` có kỳ trả góp quá hạn **> 90 ngày** (chuẩn phân loại phổ biến kiểu Basel) → chuyển `status='defaulted'` .
3. Mỗi sổ/khoản vay xử lý **cô lập bằng try/catch riêng** — 1 lỗi không chặn cả batch (nguyên tắc resilient của xử lý hàng loạt).
4. Ghi log đầy đủ vào `cob_runs` (số lượng đã xử lý, số lỗi, chi tiết JSON) để audit.

METHOD	ENDPOINT
POST	<code>/cob/run</code>
GET	<code>/cob/runs</code>
GET	<code>/cob/runs/:id</code>

4.13 Giao diện quản trị (Web Console) — FR-UI

Trang đơn (`public/index.html`), phong cách "core banking terminal": sidebar module, breadcrumb, đồng hồ hệ thống, bảng dữ liệu dạng lưới kẻ ô, toàn bộ nhãn/label tiếng Việt (kể cả dịch giá trị enum sang tiếng Việt trong badge trạng thái). Các module hiển thị: Tổng quan, Khách hàng, Tài khoản, Tiền gửi, Thẻ, Giao dịch, Khoản vay, Cảnh báo gian lận, Sổ cái (GL), Duyệt lệnh (Maker-Checker, có form đăng nhập JWT riêng cho vai trò Maker/Checker để mô phỏng 2 người trong 1 trình duyệt), Batch cuối ngày (COB), Danh mục (account types, categories, merchant, tỷ giá + quy đổi FX).

4.14 Tài liệu API — FR-DOC

OpenAPI 3.0.3 tự sinh (`src/config/openapi.ts`), phục vụ qua Swagger UI tại <http://localhost:3000/docs>, bao phủ toàn bộ 14 module / 55 endpoint ở mục 4.1–4.13 kèm schema request/response, mã lỗi, và `securitySchemes: bearerAuth` cho các endpoint yêu cầu đăng nhập.

5. Yêu cầu phi chức năng

5.1 Bảo mật

MÃ	YÊU CẦU
NFR-SEC-01	Mật khẩu người dùng nội bộ băm bằng bcrypt (cost 10), không bao giờ lưu/trả plaintext
NFR-SEC-02	Đăng nhập cấp JWT ký bằng secret riêng (<code>JWT_SECRET</code> , không hard-code, không commit)
NFR-SEC-03	Danh tính maker/checker chỉ xác định qua token đã xác thực (<code>req.user</code>), không tin request body
NFR-SEC-04	Phân quyền RBAC thực thi ở tầng Service (không chỉ kiểm tra tồn tại user)
NFR-SEC-05	Response API không bao giờ trả <code>password_hash</code> (repository dùng danh sách cột tường minh, không <code>SELECT *</code> cho endpoint public)

5.2 Toàn vẹn dữ liệu

MÃ	YÊU CẦU
NFR-DATA-01	Mọi nghiệp vụ nhiều bước (mở sổ, trả góp, đảo giao dịch...) chạy trong 1 DB transaction (<code>withTransaction</code>) — hoặc thành công toàn bộ, hoặc rollback toàn bộ
NFR-DATA-02	Row-lock (<code>SELECT ... FOR UPDATE</code>) trên account/loan/term_deposit khi đọc để ghi, tránh race condition khi nhiều request đồng thời
NFR-DATA-03	Bút toán GL luôn theo cặp cân bằng — <code>glRepository.postJournal</code> kiểm tra <code>SUM(debit) = SUM(credit)</code> trước khi ghi, lệch → ném lỗi, không ghi nửa vời
NFR-DATA-04	Idempotency-Key bắt buộc cho mọi POST di chuyển tiền — chống double-post khi client retry

5.3 Khả năng kiểm toán (Auditability)

- Mọi bút toán GL (`gl_entries`) tham chiếu ngược về `transaction_id` gốc.
- Mọi kỳ trả góp vay (`loan_payments`) và lần ghi nhận lãi sổ tiết kiệm (`term_deposit_postings`) tham chiếu `transaction_id` thật.
- Mọi thao tác qua Maker-Checker lưu đầy đủ `maker_id` , `checker_id` , `input_at` , `decided_at` , `result` / `reject_reason` .
- Mọi lần chạy COB lưu log riêng (`cob_runs`) không ghi đè.

5.4 Khả năng bảo trì / mở rộng

- Business logic tập trung 100% ở tầng Service — Repository chỉ chứa SQL, Controller chỉ chuyển đổi request/response.
- Migration đánh số tuần tự, idempotent, không sửa đổi migration cũ.
- Domain type (`src/types/domain.ts`) là nguồn sự thật duy nhất cho hình dạng dữ liệu dùng chung toàn bộ tầng Service/Repository.

5.5 Hạn chế đã biết

- Chưa có refresh token / thu hồi phiên đăng nhập (JWT hết hạn tự nhiên theo thời gian, không revoke được giữa chừng).
- Chỉ hỗ trợ 1 chi nhánh/1 công ty, không đa tiền tệ đầy đủ (chỉ VND ↔ 1 ngoại tệ mỗi lần quy đổi).
- COB chạy trên cùng process với API server (không tách batch server riêng như hệ thống production thật).
- `POST /auth-queue/:id/authorize` và `/reject` không có Idempotency-Key riêng — hiện được bảo vệ gián tiếp bởi chính trạng thái `pending` → `authorized/rejected` của hàng đợi (không thể tác động 2 lần lên cùng 1 `queue_id`), nhưng đây là cơ chế khác bản chất so với Idempotency-Key thật.

6. Giao diện ngoài

6.1 REST API

- Base URL: `http://localhost:3000/api/v1`
- Envelope response chuẩn cho **mọi** endpoint:

```
{ "status": "SUCCESS" | "ERROR", "data": <object|array|null>, "error": { "code": "...", "message": "..."} | null, "meta?": { "total": n, "page": n, "size": n } }
```

- Danh sách đầy đủ: mục 4.1–4.14 ở trên (14 module, 55 endpoint).

6.2 Swagger UI (tài liệu API tương tác)

- URL: <http://localhost:3000/docs>

- Tự sinh từ `src/config/openapi.ts` (OpenAPI 3.0.3) — cho phép thử trực tiếp từng endpoint kèm mẫu request/response, không cần Postman/curl.

6.3 Web Console

- URL: `http://localhost:3000/`
- Không cần build step — HTML/CSS/JS thuần, gọi thẳng REST API cùng origin.

7. Phụ lục

7.1 Cấu trúc thư mục chính

<code>db/schema.postgres.sql</code>	11 bảng nghiệp vụ gốc
<code>db/migrations/</code>	001-007, chạy tuần tự qua <code>scripts/migrate.mjs</code>
<code>src/repositories/</code>	1 file / bảng – SQL thuần qua pg
<code>src/services/</code>	business rules – tầng DUY NHẤT chứa logic nghiệp vụ
<code>src/controllers/ + routes/</code>	khớp từng endpoint
<code>src/middleware/</code>	<code>validate (zod)</code> , <code>requireAuth (JWT)</code> , <code>requireIdempotencyKey</code> , <code>errorHandler</code>
<code>src/jobs/cobScheduler.ts</code>	cron job COB
<code>src/config/openapi.ts</code>	đặc tả OpenAPI dùng cho Swagger UI
<code>public/index.html</code>	giao diện quản trị

7.2 Biến môi trường (`.env`)

BIẾN	MẶC ĐỊNH	MỤC ĐÍCH
<code>PORT</code>	3000	Cổng HTTP
<code>DATABASE_URL</code>	—	Connection string Postgres
<code>DB_SSL</code>	true	Bật SSL cho Supabase
<code>FRAUD_AMOUNT_THRESHOLD</code>	50,000,000	Ngưỡng VND tự tạo fraud alert
<code>JWT_SECRET</code>	(bắt buộc đặt riêng)	Khoá ký JWT
<code>JWT_EXPIRES_IN</code>	8h	Thời hạn token
<code>COB_CRON_SCHEDULE</code>	<code>00 * * *</code>	Lịch chạy COB tự động

7.3 Việc còn mở (đề xuất mở rộng tiếp theo)

- Refresh token + danh sách thu hồi (session revocation).
- Đa chi nhánh / đa công ty (`branch_id` xuyên suốt accounts, GL, báo cáo).
- Đa tiền tệ đầy đủ (GL có chiều tiền tệ, revaluation cuối kỳ).
- Tách batch server COB khỏi API server (queue/worker riêng).

5. Giao diện khách hàng (self-service banking).